v1.7 块级检索优化与重排实验记录

项目：robot-vacuum-agent（扫地机器人智能客服 Agent） | 模型：deepseek-chat（agent）+ deepseek-reasoner（judge）
判据版本：j5 | 数据集：30 条（含 16 条检索类） | 记录时间：2026-11-13

一、本轮目标

v1.6 之后唯一系统性失败是 R012「地面有水能清扫吗」（检索未命中）。但仅凭"通过率"无法判断问题出在召回还是排序，因此本轮先**把检索判据从文件级细化到块级**，再据此决定优化方向。

二、做了什么

改动	内容
块级判据	为 16 条检索类用例补 `gold_chunk`（关键句原文），新增 `chunk_recall@3`、`chunk_recall@5`、`chunk_MRR@5` 三个指标
多轮检索查询	多轮用例的检索查询改为合并历史用户发言（原来只拿最后一句"那选哪个型号？"，无法检索）
重排模块	新增 `rag/reranker.py`：BM25（jieba 分词）+ 向量召回，RRF 融合；通过 `chroma.yml` 的 `rerank_enabled` 开关控制
评测开关	`run_eval.py` 增加 `--rerank / --no-rerank`，默认跟随配置
工程	`factory.py` 的 `chat_model` 改懒加载：`--no-llm` 检索评测不再要求对话模型 key

三、修了 4 个评测 bug（都影响过之前的数字）

Bug	现象 / 根因	修复与影响
多轮查询只取最后一句	M001「那选哪个型号？」单独检索无意义，被判"未命中"，实际是评测写错	合并历史用户发言后：M001 由 **未命中** → **块排名第 2**
`@5` 指标被 `k=3` 限死	检索指标复用了 retriever 的 `k=3`，所谓 recall@5 实际只看 3 条	改为直接向向量库取 top-k：recall@5 由 87.5% → **100%**（真实值）
重排后拿不到向量库	开启重排时 retriever 变成 `RunnableLambda`，`retriever.vectorstore` 不存在，指标静默退化为"只看 3 条"	指标改为直接持有 Chroma 实例，不再依赖 retriever 类型
重排开关写死	`rerank=not args.no_rerank` 与配置无关，全量跑时意外启用重排	改为跟随 `chroma.yml`，并提供强制开关

四、块级基线（k=3，无重排）

指标	数值	含义
hit@3（关键词覆盖）	93.8%	top-3 能否覆盖关键事实
src@1 / src@3	75% / 100%	正确*文件*的排名
MRR@5（文件级）	0.865	文件排序质量
chunk_recall@3	**87.5%**	正确*块*是否进 top-3
chunk_recall@5	**100%**	正确块是否进 top-5
chunk_MRR@5	0.841	块排序质量

关键判断：正确块 100% 在 top-5 内，但只有 87.5% 进 top-3 —— 这是**排序问题**，不是召回问题。因此结论是：优先做重排，而不是改 chunk_size 或换 embedding 模型。

五、实验一：本地 BM25 + RRF 重排（结论：无效）

方案：向量召回 top-10 → 用 BM25（jieba 分词，全库 88 块统计 idf）对候选再排序 → RRF 融合（`score = 1/(60+vec_rank) + α·1/(60+bm25_rank)`）→ 取 top-k。

α（BM25 权重）	chunk_recall@3	chunk_recall@5	chunk_MRR@5
0.0（纯向量，基线）	87.5%	100%	**0.841**
0.1	87.5%	100%	0.841
0.2	87.5%	100%	0.809
0.3	87.5%	100%	0.714
0.5	81.2%	100%	0.624
1.0	68.8%	100%	0.453

结论：**任何 BM25 权重都会降低排序质量**，α=0 最优。原因：知识库只有 88 块且全部围绕同一主题，BM25 的 idf 区分度不足，关键词命中会"劫持"本已正确的向量排序。

为什么不继续硬调：这是"方法不适用"，不是"参数没调好"。继续调 α 只会过拟合 16 条用例。重排需要语义模型（cross-encoder），而不是词频统计。

六、实验二：k=3 → k=5（结论：召回换精度）

既然正确块 100% 在前 5，直接把 `k` 提到 5，让模型直接"看见"这 5 块。

指标	k=3（v1.6）	k=5（v1.7）	变化
通过率	96.7%	96.7%	持平
检索类通过率	93.3%	**100%**	**+6.7pp（R012 修复）**
块级 chunk_recall@5	100%	100%	持平
忠实度	**66.7%**	**43.3%**	**-23.4pp**
相关性均分	4.10	3.93	-0.17
工具调用 / 参数	100% / 100%	100% / 100%	持平
安全 / 拒答	100% / 100%	100% / 100%	持平
平均延迟	4.99s	4.29s	-0.70s
P95 延迟	15.17s	11.38s	-3.79s
输入 / 输出 tokens	2537 / 327	2401 / 333	基本持平
单次成本	0.00769 元	0.00746 元	-3%

解读：k=5 修好了 R012（检索类 100%），但**忠实度从 66.7% 掉到 43.3%**——多召回的 2 块里混入了无关或相似但不适用的内容，模型把它们也当成依据，产生"有出处但答偏"的表述。这是典型的**召回-精度权衡**，不能只看通过率。

七、本轮唯一失败

用例	层次	现象
T003「今天适合拖地吗」	检索/回答层	天气工具返回空（外部源不可用），模型遵守"不得编造"给了保守回答，关键词判据未命中

属外部依赖问题，非模型能力问题；下一轮给天气工具加重试与降级提示。

八、结论与下一步

1. **重排仍然是对的，但必须是语义重排**：本地 BM25 已证伪，等账号开通 `gte-rerank` 后接入（接口已按同一签名预留，改一行即可切换）。
2. **k=5 只是权宜之计**：它用"多看两块"换来了检索类 100%，代价是忠实度 -23.4pp。若开通重排，应回到 k=3 并靠重排把正确块顶到第 1。
3. **块切分值得下一步做**：知识库是"问题 + 答：" 结构，按问答边界切分（而非固定 200 字）可让"事实句"整块保留，预计同时改善召回与忠实度。
4. **查询改写**：R005/R012 这类口语化提问（"地面有水"）与文档表述（"不防水""进水"）不匹配，值得加同义词扩展。

九、复现命令

```
cd /d D:\开发app\pycharm\Agent项目
set PYTHONPATH=%CD%

:: 只跑检索层（不消耗对话额度，几秒出结果）
.venv\Scripts\python.exe eval\run_eval.py --no-llm
.venv\Scripts\python.exe eval\run_eval.py --no-llm --rerank     :: 本地 BM25+RRF
.venv\Scripts\python.exe eval\run_eval.py --no-llm --no-rerank  :: 纯向量

:: 全量（agent=deepseek-chat，judge=deepseek-reasoner）
.venv\Scripts\python.exe eval\run_eval.py --judge --judge-provider deepseek --judge-model deepseek-reasoner
--no-rerank --save-baseline
```

十、指标口径说明

- **hit@3**：top-3 块的并集是否覆盖 `must_contain` 关键事实（关键词匹配）。
- **src@k**：top-k 中是否出现 `gold_source` 指定的文件（文件级召回）。
- **chunk_recall@k**：top-k 中是否有块包含 `gold_chunk` 关键句（块级召回）。
- **chunk_MRR@5**：首个命中块排名的倒数均值（排序质量）。
- **忠实度**：LLM-as-Judge 判断回答事实是否均有检索依据；**相关性**：1–5 分，≥3.5 记通过。